

TECHNICAL ARCHITECTURE DOCUMENT

Backend Integration & Architecture Migration

Removing the BFF mock layer and wiring the Next.js/React frontend directly into the Core NestJS Backend Engine across all modules.

Scope

M02, M04, M08,
M09+

Type

Production
Migration

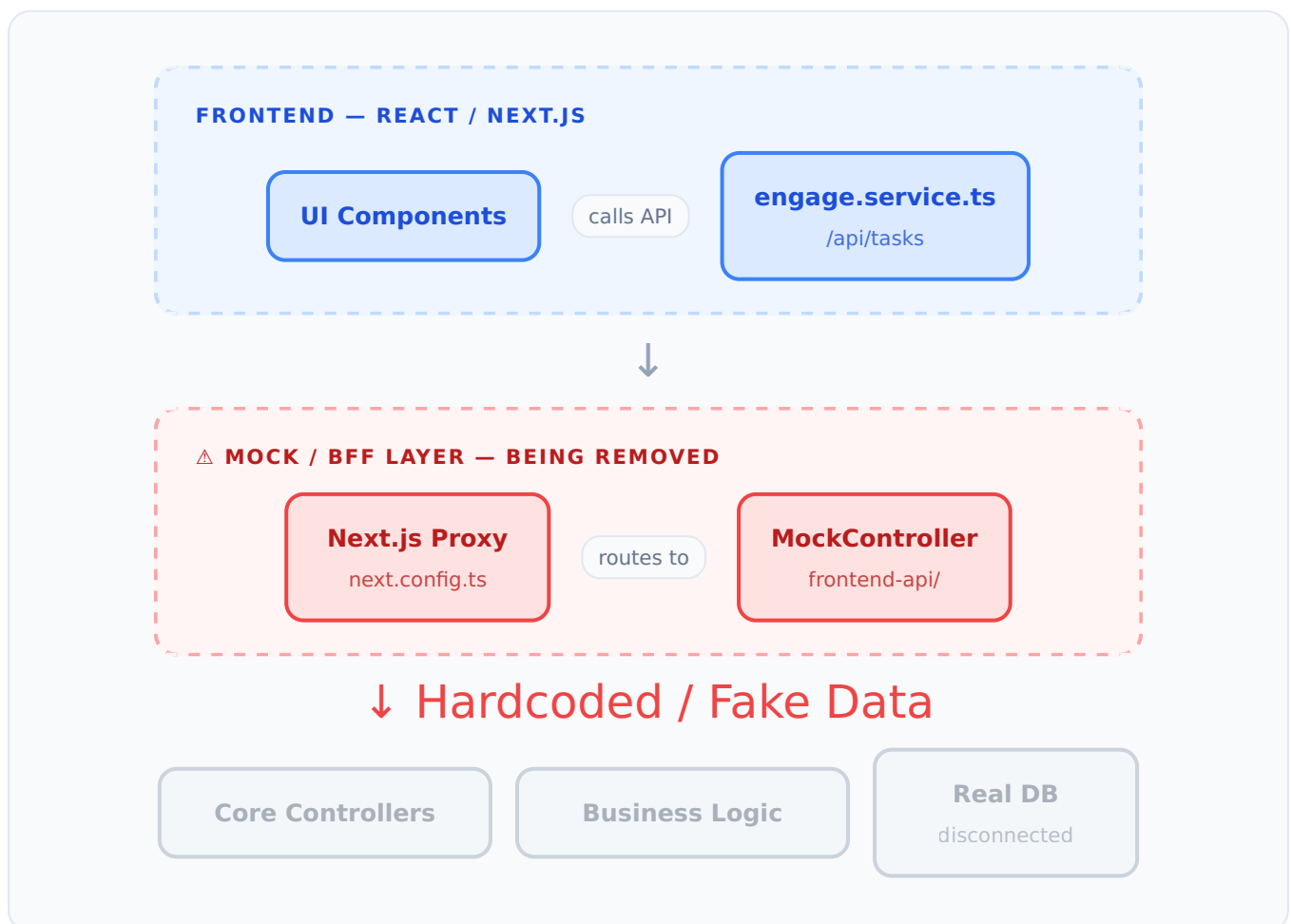
Status

In
Progress

GOAL

Remove the temporary "Backend-for-Frontend" (BFF) mock layer and wire the Next.js/React frontend directly into the **Core NestJS Backend Engine** and its real database tables.

BEFORE — THE PROTOTYPE FACADE



FRONTEND — REACT / NEXT.JS

UI Components

calls API

engage.service.ts

/api/v1/sales-engagement

↓ Direct — No Proxy

✓ REAL CORE BACKEND

Core Controllers

/api/v1/...

→

Business Logic

→

Real DB

Tasks, SalesPlays

2 Universal Migration Workflow

This workflow applies to **every module** across the monorepo (M02, M04, M08, M09, ...). Follow these steps in strict order — always **Backend-First**.

1 Identify the Data Gap & Schema Mismatches

Determine what the React UI expects (e.g., `conversionRate`, `meddpiccScore`). Check whether the core backend already provides it natively — if not, document the gap precisely before touching any code.

2 Upgrade the Core Engine (Backend)

Database: Modify `schema.prisma` if new columns are needed. **Logic:** Update the Core Service and Repositories to run real SQL aggregations. **Mappers:** Add mapping logic inside the core NestJS Service to transform the Prisma model shape to what the frontend components expect.

3 Standardize the Core Controller

Ensure the Core Controller (e.g., `m08.controller.ts`, `deals.controller.ts`) exposes endpoints under the standard `/api/v1/[module-name]` path structure.

4 Rewire the Frontend & Remove Proxies

Update frontend service files to call `resolveApiBase() + '/api/v1/...'`. Delete the old mock controllers from the backend. Remove the old rewrite rules from `unified-ui/next.config.ts`.

Migration Checklist

Task	File / Location	Action
Remove proxy rewrite	<code>unified-ui/next.config.ts</code>	Remove
Delete mock controller	<code>frontend-api/manager/...controller.ts</code>	Delete
Delete mock service	<code>frontend-api/manager/...service.ts</code>	Delete
Update Prisma schema	<code>schema.prisma</code>	Update
Add mapper logic	Core NestJS Service	Add
Rewire frontend service (rep)	<code>features/engage/rep/services/...</code>	Update
Rewire frontend service (manager)	<code>features/engage/manager/services/...</code>	Update

Edge Cases & Important Nuances

During migration you will encounter several recurring edge cases. Always check for these before starting a module migration.

EDGE CASE A

Next.js Proxy Rewrite Scope

Some modules intercept broad paths like `/api/deals/:path*`; others intercept highly specific ones like `/api/tasks/summary`. Audit `next.config.ts` carefully and remove *only* the rewrites for the current module — don't break unmigrated modules.

EDGE CASE B

Frontend Service Naming Variations

There is no single naming convention. M04 uses `dealBoardsService.ts`, M08 uses `engage.service.ts`, M02 uses `call-reviews.service.ts`. Use full-text search for the old `/api/` route strings to locate the right files.

EDGE CASE C

Rep vs. Manager Bifurcation

Many modules have completely separate frontend codebases for Sales Reps vs. Sales Managers. Always check for both personas and rewire *both* service files.

EDGE CASE D

"Missing Table" vs "Missing Field"

The BFF layer may query a real Prisma model (like `EngageTask`) rather than an in-memory mock. Don't delete Prisma models arbitrarily — first check if the core tables (`Task`, `SalesPlay`) can absorb all data requirements.

CONCRETE EXAMPLE

The following is an exact trace of how a single request flowed through the old Bridge layer using **M08 Sales Engagement** as the reference module.

Step 1 React UI makes a request

`unified-ui/src/features/engage/components/manager/services/engage.service.ts`

```
// Fetches from a generic relative path
const endpoint = '/api/tasks?assigneeId=me';
const res = await apiFetch('GET', endpoint);
```

Step 2 Next.js Proxy silently rewrites the URL

`unified-ui/next.config.ts`

```
async rewrites() {
  return [
    {
      source: '/api/tasks/:path*',
      // Silently redirects to port 3008 (M08 microservice)
      destination: 'http://localhost:3008/api/tasks/:path*',
    },
  ];
}
```

Step 3 Mock Controller receives the request

`modules/m08-sales-engagement/frontend-api/manager/m08-frontend-engage-manager.controller.ts`

```
// Lives in a folder called "frontend-api" – this is the mock layer
@Controller()
export class M08FrontendEngageManagerController {
  @Get('api/tasks')
  tasks(@Query('assigneeId') assigneeId: string) {
    return this.svc.fetchTasks(assigneeId);
  }
}
```

Step 4

Bridge Service queries the fake EngageTask table

modules/m08-sales-engagement/frontend-api/manager/m08-frontend-engage-manager.service.ts

```
async fetchTasks(assigneeId: string) {
  //   Queries 'EngageTask' – a simplified mock table
  //   Ignores the real 'Task' and 'SalesPlay' models entirely
  return this.prisma.engageTask.findMany({
    where: { assigneeId }
  });
}
```

The Fix — After Applying the Migration Workflow

Action	Detail
Delete proxy rule	Remove /api/tasks/:path* entry from next.config.ts
Delete mock controller	Remove M08FrontendEngageManagerController
Update frontend service	Change to resolveApiBase() + '/api/v1/sales-engagement/tasks'
Use real tables	Core engine queries real Task + SalesPlay models